

Number Systems

15 May 2026 05:12

How are numeric data items actually stored in the computer memory?

How much space(memory locations) is allocated for each type of data?

- int, float, char etc.

How are characters and strings are stored in the memory?

Number System:: The Basics

We are accustomed to using the so called **decimal number system**.

- Ten digits::0,1,2,3,4,5,6,7,8,9
- Every digit position has a weight which is A power of 10.
- Base or radix is 10.

Example:

$$234 = 2 \cdot 10^2 + 3 \cdot 10^1 + 4 \cdot 10^0$$

$$250.67 = 2 \cdot 10^2 + 5 \cdot 10^1 + 0 \cdot 10^0 + 6 \cdot 10^{-1} + 7 \cdot 10^{-2}$$

Binary Number System

Two digits:

- 0 and 1
- Every digit position has a weight which is A power of two.
- Base or radix is 2.

Example:

$$110 = 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$$

$$101.01 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2}$$

Binary to Decimal conversion

Each digit position of a binary number has a weight.

- Some power of two

A binary number:

$$B = b_{n-1}b_{n-2}...b_1b_0b_{-1}b_{-2}...b_{-m}$$

Corresponding value in decimal:

$$D = \sum_{i=-m}^{n-1} b_i \cdot 2^i$$

Examples:

$$\begin{array}{c} 5 \quad 4 \quad 3 \quad 2 \quad 1 \quad 0 \\ 101011 \end{array} \rightarrow 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$
$$= 32 + 0 + 8 + 0 + 2 + 1$$
$$= 43_{10}$$

$$\begin{array}{c} 0 \quad 1 \quad 0 \quad 1 \\ 0.0101 \end{array} \rightarrow 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4}$$
$$= 0 + \frac{1}{4} + 0 + \frac{1}{16} = \frac{4+1}{16} = \frac{5}{16} = 0.3125$$

$$= 0 + \frac{1}{4} + 0 + \frac{1}{16} = \frac{4+1}{16} = \frac{5}{16} = 0.3125$$

$$0.0101_2 = 0.3125_{10}$$

$$\begin{aligned} 101.11_2 &= 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} \\ &= 4 + 0 + 1 + 0.5 + 0.25 \\ 101.11_2 &= 5.75_{10} \end{aligned}$$

Decimal-to-Binary Conversion

Consider the integer and fractional parts separately

For integer part,

- Repeatedly divide the given number by two and go on accumulating the remainders until the number becomes 0.
- Arrange the remainders in reverse order.

For the fractional part:

- Repeatedly multiply the given fraction by two.
 - Accumulate the integer part (0, 1).
 - If the integer part is one, chop it off.
- Arrange the integer part in order they are obtained.

Examples:

$$(239)_{10} = (?)_2$$

2	239	
2	119	1
2	59	1
2	29	1
2	14	1
2	7	0
2	3	1
2	1	1
	0	1

$$(239)_{10} = (11101111)_2$$

2 ²	2 ¹	2 ⁰
4	2	1

0	1	1	3
1	0	1	5

5	4	3	2	1	0
32	16	8	4	2	1

$$\begin{aligned} (1110001)_2 \\ (1010001)_2 \end{aligned}$$

$$\begin{aligned} & \dots 0 \text{ or } 1 \\ & \left. \begin{array}{l} 10 \\ 00 \\ 11 \\ 01 \end{array} \right\} \\ & 0+7 \\ & 2^6 - 1 \\ & 2^n - 1 \\ & 2^2 - 1 \end{aligned}$$

11	10	9	8	7	6	5	4	3	2	1	0
2048	1024	512	256	128	64	32	16	8	4	2	1

$$0.634$$

0	634	x 2
1	268	x 2
0	536	x 2
1	072	x 2

$$(0.634)_{10} = (0.01010\dots)_2$$

0	536	$\times 2$
1	072	$\times 2$
0	144	
$\vdots$	$\vdots$	

~ 10

~ 12

$$(37.625)_{10} = (\quad ? \quad)_2$$

$$37_{10} = 100101_2$$

$$37.625_{10} = 100101_2 + 0.101_2$$

$$= 100101.101_2$$

0	625 $\times 2$
1	250 $\times 2$
0	500 $\times 2$
$\downarrow$ 1	000

3216 8421
100101

Hexadecimal Number System

A compact way of representing binary numbers

16 different symbols (radix = 16)

4 digits.

$$2^3 = 8$$

0000	$\rightarrow$ 0
0001	$\rightarrow$ 1
0010	$\rightarrow$ 2
0011	$\rightarrow$ 3
0100	$\rightarrow$ 4
0101	$\rightarrow$ 5
0110	$\rightarrow$ 6
0111	$\rightarrow$ 7
1000	$\rightarrow$ 8
1001	$\rightarrow$ 9
1010	$\rightarrow$ 10 or A
1011	$\rightarrow$ 11 or B
1100	$\rightarrow$ 12 or C
1101	$\rightarrow$ 13 or D
1110	$\rightarrow$ 14 or E
1111	$\rightarrow$ 15 or F

0000	$\leftarrow$
0001	$\leftarrow$ 6
0010	$\leftarrow$ 0
0011	$\leftarrow$ 0
0100	$\leftarrow$ 0
0101	$\leftarrow$ 0
0110	$\leftarrow$ 0
0111	$\leftarrow$ 0
1000	$\leftarrow$ 1
1001	$\leftarrow$ 1
1010	$\leftarrow$ 1
1011	$\leftarrow$ 1
1100	$\leftarrow$ 1
1101	$\leftarrow$ 1
1110	$\leftarrow$ 1
1111	$\leftarrow$ 1

Binary to Hexadecimal Conversion

For the integer part:

- Scan the binary number from right to left.
- Translate each group of four bits into the corresponding hexadecimal digit.
 - Add leading zeros if necessary.

For the fractional part:

- Scan the binary number from left to right.
- Translate each group of four bits into the corresponding hexadecimal digit.
 - Add trailing zeros if necessary.

$$(010110100011)_2 = (5A3)_{16}$$

$$(0101.0100011)_2 = (5A3)_{16}$$

$\downarrow \quad \downarrow \quad \downarrow$
 $5 \quad 10 \quad 3$

$$(0.10000100)_2 = (0.84)_{16}$$

$\downarrow \quad \downarrow$
 $8 \quad 4$

$$(0101.0101110)_2 = (5.5E)_{16}$$

$\downarrow \quad \downarrow \quad \downarrow$
 $5 \quad 5 \quad E$

Hexadecimal to binary Conversion

Translate every hexadecimal digit into its 4 bit binary equivalent.

Examples :

$$(3AS)_{16} = (1110100101)_2$$

$$(12.3D)_{16} = (00010010.0011101)_2$$

8	4	2	1
0	1	0	1
1	0	1	0
1	1	0	1
0	0	1	1
0	0	1	0

Octal Number System

A compact way of representing binary numbers

8 different symbols (radix = 8)

000	→	0
001	→	1
010	→	2
011	→	3
100	→	4
101	→	5
110	→	6
111	→	7

Binary to Octal Conversion

For the integer part:

- Scan the binary number from right to left.
- Translate each group of three bits into the corresponding hexadecimal digit.
 - Add leading zeros if necessary.

For the fractional part:

- Scan the binary number from left to right.
- Translate each group of three bits into the corresponding hexadecimal digit.
 - Add trailing zeros if necessary.

/ ← ← ←

- Translate each group of three bits into the corresponding hexadecimal digit.
 - Add trailing zeros if necessary.

$$\begin{aligned} (01010101)_2 &= (1255)_8 \\ (.010101110)_2 &= (0.256)_8 \end{aligned}$$

Octal to binary Conversion

Translate every hexadecimal digit into its 3 bit binary equivalent.

$$\begin{aligned} (437)_8 &= (100\ 011\ 111)_2 \\ (53.36)_8 &= (101\ 011.011\ 110)_2 \end{aligned}$$

WAP] to convert from one number system to another number system. Your program will generate a menu:

```
Press 1 Convert from decimal to binary
Press 2 Convert from binary to decimal
Press 3 Convert from decimal to hexadecimal
Press 4 Convert from hexadecimal to decimal
Press 5 Convert from hexadecimal to binary
Press 6 Convert from binary to hexadecimal
Press 7 Convert from binary to octal
Press 8 Convert from octal to binary
Press 9 Convert from decimal to octal
Press 10 Convert from octal to decimal
Press 11 Convert from octal to hexadecimal
Press 12 Convert from hexadecimal to octal
Enter your choice:
```

Unsigned Binary Numbers

A n-bit binary number

$$B = b_{n-1}b_{n-2}\dots b_2b_1b_0$$

2^n distinct combinations are possible, 0 to $2^n - 1$

For example, for $n = 3$, there are 8 distinct combination

000, 001, 010, 011, 100, 101, 110, 111

Range of numbers that can be represented

n = 8	0 to 2^8-1 (255)
n = 16	0 to $2^{16}-1$ (65535)
n = 32	0 to $2^{32}-1$ (4294967295)

Signed Integer Representation

Many of the numerical data items that are used in a program are signed (positive or negative).

Question:: How to represent sign?

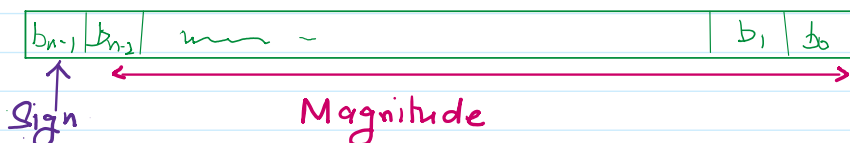
Three possible approaches:

- Sign-magnitude representation
- One's complement representation
- Two's complement representation

Sign-magnitude Representation

For an n-bit representation

- The most significant bit (MSB) indicates sign.
 - 0 -> positive
 - 1 -> negative
- The remaining (n - 1) bits represents magnitude.



Range of numbers that can be represented:

Maximum :: $+(2^{n-1} - 1)$

Minimum :: $-(2^{n-1} - 1)$

$$\begin{aligned} n &= 5 \\ + (2^{4-1}) &= 15 \\ - (2^{4-1}) &= -15 \end{aligned}$$

A problem:

Two different representation of zeros:

+0 -> 0 000...0

-0 -> 1 000...0

One's Complement Representation

Basic idea:

- Positive numbers are represented exactly as in sign-magnitude form.
- Negative numbers are represented in one's complement form.

How to compute the ones complement of a number?

- Complement every bit of the number (1 -> 0 and 0 -> 1)
- MSB will indicate the sign of the number
 - 0 -> positive
 - 1 -> negative

Example:: n = 4

0000 -> +0

0001 -> +1

0010 -> +2

0011 -> +3

1000 -> -7

1001 -> -6

1010 -> -5

To find the representation of

0000 -> 0
 0001 -> +1
 0010 -> +2
 0011 -> +3
 0100 -> +4
 0101 -> +5
 0110 -> +6
 0111 -> +7

1000 -> -7
 1001 -> -6
 1010 -> -5
 1011 -> -4
 1100 -> -3
 1101 -> -2
 1110 -> -1
 1111 -> -0

To find the representation of
 say -4, first note that
 $+4 = 0100$
 $-4 = 1011 \leftarrow 1's \text{ complement.}$

Range of numbers that can be represented:

Maximum :: $+(2^{n-1} - 1)$

Minimum :: $-(2^{n-1} - 1)$

A problem:

Two different representation of zeros:

+0 -> 0 000...0

-0 -> 1 111...1

Advantages of 1's complement representation

- Subtraction can be done using addition
- Leads to substantial saving in circuitry

Subtraction using addition:: 1's complement

How to compute $A - B$?

- Compute the 1's complement of B (say B_1)
- Compute $R = A + B_1$
- If the carry is obtained after addition is '1'
 - Add the carry back to R (called end-around carry).
 - That is $R = R + 1$
 - The result is a positive number
- Else
 - The result is negative and is in 1's complement form.

Example 1 :: $6 - 2$

Solution :: 4-bit representation.

Binary of 2 = 0010

1's complement of 2 = 1101

Binary of 6 = 0110

6 :: 0110

- 2 :: 1101

1001

+1

0100

(+4)

↑

$R = A - B$, $B_1 = B'$

A

B_1

$R = A + B'$

Example 2 :: $7 - 6$

6 bits

14 12 10 8 6 4 2 1

Example 2:: $20 - 60$

Solution:

Binary of 60 = 0111100

1's complement of 60 = 1000011 (B_1)

Binary of 20 = 0010100 (A)

$$R = A + B_1$$

$$\begin{array}{r} 0010100 \quad A \\ + 1000011 \quad B_1 \\ \hline 1010111 \quad (-40) \end{array}$$

Since there is no carry, the result is negative.

Example:: $-20 - 60$ (Both of them are negative)

Binary of 20 = 0010100

1's complement of 20 = 1101011 (A)

Binary of 60 = 0111100

1's complement of 60 = 1000011 (B_1)

$$R = A + B_1$$

$$\begin{array}{r} 1101011 \\ + 1000011 \\ \hline 10101110 \end{array}$$

$$\begin{array}{r} (-) 0101111 \end{array}$$

both the values are (-ve)

$$\begin{array}{r} 64 \ 32 \ 16 \ 8 \ 4 \ 2 \ 1 \\ 0 \ 1 \ 0 \ 1 \ 1 \ 1 \ 1 \\ 1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \leftarrow 80 \end{array}$$

For example

$$-6 + 2$$

1's complement of (-6) only & follow the process.

Overflow: $5 + 6$ in 4 bit

$$\begin{array}{l} 5_{10} = (0101)_2 \\ 6_{10} = (0110)_2 \end{array}$$

$$\begin{array}{r}
 0101 \\
 + 0110 \\
 \hline
 1011
 \end{array}$$

-8	4	2	1
1	0	1	1

negative number in 1's complement
 Overflow.
 Overflows are handled separately.

How to handle the problem of two 0's?

Two's Complement Representation

Basic idea:

- Positive numbers are represented exactly as in sign - magnitude form.
- Negative numbers are represented in 2's complement form.

How to compute the two's complement of a number?

- Complement every bit of a number (1 $\rightarrow$ 0 and 0 $\rightarrow$ 1), and then add one to the resulting number.
- MSB will indicate the sign of the number.
 - ♦ 0 $\rightarrow$ positive
 - ♦ 1 $\rightarrow$ negative

Example: 6 - 2 binary using 2's complement

Solution: 4-bit binary

binary of 6 = 0110 — (A)

binary of 2 = 0010

1's complement of 2 = 1101

2's complement of 2 = 1101

$$\begin{array}{r}
 1101 \\
 + 1 \\
 \hline
 1110
 \end{array}
 \quad \text{— (B)}$$

$R = A + B$

$$\begin{array}{r}
 0110 \\
 + 1110 \\
 \hline
 10100
 \end{array}$$

discard and the result is (0100)

Range of numbers that can be represented:

Maximum :: $+(2^{n-1} - 1)$

Minimum :: $-(2^{n-1})$

Advantage:

- Unique representation of zero.
- Subtraction can be done using addition
- However leads to substantial saving in circuitry

Almost all computers today use the two complement representation for storing negative numbers.

} descending order of marks

Subtract using 2's complement

$$(i) (10110)_2 - (1000111)_2$$

$$A = (0010110)_2$$

$$B = (1000111)_2$$

$$B_1 = (0111000)_2 \leftarrow \text{1's complement}$$

$$\begin{array}{r} 0111000 \\ +1 \\ \hline 0111001 \end{array} \leftarrow \text{2's complement}$$

$$\begin{array}{r} 0010110 \\ 0111001 \\ \hline 1001111 \end{array}$$

MSB is 1 hence the value is -ve

9's complement: $27 - 16$

$$B_1 \rightarrow \begin{array}{r} 99 \\ -16 \\ \hline 83 \end{array}$$

$$\begin{array}{r} 27 \\ 83 \\ \hline 110 \\ +1 \\ \hline 111 \end{array}$$

$$\begin{array}{r} 111 \\ 0101 \\ \hline 1010 \end{array}$$

FFFF

10's complement $27 - 16$

$$B_1 \rightarrow \begin{array}{r} 99 \\ -16 \\ \hline 83 \\ +1 \\ \hline 84 \end{array} \leftarrow \begin{array}{l} \text{9's complement} \\ \text{10's complement} \end{array}$$

27
84

X | |
↑
discard.

$$\begin{array}{r} 99 \\ 88 \\ \hline 11 \end{array}$$
$$\begin{array}{r} 99 \\ 27 \\ \hline 72 \end{array}$$

$$\begin{array}{r} 16 \\ + 72 \\ \hline 88 \end{array}$$
 (This is 10's complement value, no carry means value is -ve.)
 $(-12)_{10}$

$$\begin{array}{r} A2DB_{16} + C953_{16} \\ \overset{1}{A}2DB \\ + C953 \\ \hline 15C2E \end{array}$$

' 0 9 A B C D E F '

↑ ↑
11 12

16 | 22 | 1 16 | 18 | 1

 16

 5 2

Ex: 2 : FACE₁₆ + ACE2₁₆

1 1 1
FACE
ACE2

1A7B0₁₆

$$\begin{array}{r} 12 \overline{) 271} \\ \underline{16} \\ 11 \end{array}$$

EX 3:

$$\begin{array}{r} 1A89_{16} + BAC_{16} \\ 11 \\ 1A89 \\ + BAC \\ \hline 2635_{16} \end{array}$$

Subtraction = $FACE - 1BAC$

$$\begin{array}{r} \text{FACE} \\ - 1BAC \\ \hline DEED \end{array}$$

$$\begin{array}{r} 10+6 \\ 676 \\ 68 \\ \hline 08 \end{array}$$

$$\begin{array}{r} EFACE \\ - 1BAC \\ \hline DF22_{16} \end{array}$$

$$\begin{array}{r} 68 \\ - 08 \\ \hline \end{array}$$

Ex 2: $526B_{16} - 137D_{16}$

$$\begin{array}{r} 4\cancel{1}2\overset{5}{B}27 \\ 137D \\ \hline 3EEE_{16} \end{array}$$

Ex 3: $764D_{16} - 2ADE_{16}$

$$\begin{array}{r} 6\cancel{7}\overset{5}{3}AD \\ 2ADE \\ \hline 4B6F_{16} \end{array}$$

$$\begin{array}{r} 1111 \\ -2-10 \end{array}$$

Ex 4: $1BAA_{16} - 2CBB_{16}$

$$\begin{array}{r} 1\overset{0}{A}\overset{9}{A}A \\ -14\cancel{1}\cancel{B}\cancel{A}A \\ 2CBB \\ \hline -DEEF_{16} \end{array}$$

Ex 5: $2576_{16} - 5409_{16}$

$$\begin{array}{r} -16 \\ 25\overset{6}{7}6 \\ 5409 \\ \hline -916D_{16} \end{array}$$

$$\begin{array}{r} 22 \\ -9 \\ \hline 13 \end{array}$$

Multiplication:

Ex: $25C9_{16} \times B_{16}$

$$\begin{array}{r} 381 \\ 25C9 \\ \times B \\ \hline 19FA3 \end{array}$$

$$\begin{array}{r} 16 \overline{) 63} 3 \\ \underline{48} \\ 15 \end{array}$$

$$\begin{array}{r} 8 \\ 16 \overline{) 138} \\ \underline{128} \\ 10 \end{array}$$

Ex: $2AC5_{16} \times BD_{16}$

$$\begin{array}{r} 783 \\ 2AC5 \\ \times BD \\ \hline 11XBD \\ 1AC01 \\ 1D677X \\ \hline 1F1371_{16} \end{array}$$

$$\begin{array}{r} 8 \\ 16 \overline{) 140} \\ \underline{128} \\ 12 \end{array}$$

$$\begin{array}{r} 7 \\ 16 \overline{) 118} \\ \underline{112} \\ 6 \end{array}$$

$$\begin{array}{r} 1\ D\ 6\ 7\ 7\ x \\ 1\ F\ 1\ 3\ 7\ 1_{16} \end{array}$$

Binary Multiplication:

Ex1: $1011_2 \times 11_2$

$$\begin{array}{r} 1011 \\ \times 11 \\ \hline 1011 \\ 1011x \\ \hline 100001_2 \end{array}$$

Ex2: $10110_2 \times 101_2$

$$\begin{array}{r} 10110 \\ \times 101 \\ \hline 10110 \\ 00000x \\ 10110xx \\ \hline 1101110_2 \end{array}$$

Hexadecimal Addition :

$FACE.AA_{16} + BCA.BB$

$$\begin{array}{r} 1\ 1\ 1\ 1\ 1 \\ FACE.AA \\ BCA.BB \\ \hline 10699.65_{16} \end{array}$$

Hexadecimal subtraction:

$FACE.AA - BCA.BB_{16}$

$$\begin{array}{r} \overset{D}{\cancel{F}}\overset{9}{\cancel{A}}\overset{9}{\cancel{C}}\overset{7}{\cancel{E}}.AA \\ - BCA.BB \\ \hline EF03.EF_{16} \end{array}$$

Octal Addition :

Ex: $765_8 + 126_8$

$$\begin{array}{r} 1\ 1 \\ 765 \\ + 126 \\ \hline 1112 \end{array}$$

$$\begin{array}{r} 765 \\ + 126 \\ \hline 1113_8 \end{array}$$

Octal Subtraction:

Ex: $3473_8 - 664_8$

$$\begin{array}{r} ^6 \\ 2\cancel{3}4\cancel{7}3 \\ - 664 \\ \hline 2607_8 \end{array}$$

$$\begin{array}{r} 1 \\ 8 \overline{) 11} \\ \underline{8} \\ 3 \end{array}$$

Ex: $34.55_8 - 54.67_8$

$$\begin{array}{r} -8^234 \\ 3\cancel{4}.\cancel{5}5 \\ - 54.67_8 \\ \hline -37.66_8 \end{array}$$

$$\begin{array}{r} -6-5 \\ -11 \end{array}$$

(Final carry is discarded)

Octal Multiplication

Ex $67_8 \times 11_8$

$$\begin{array}{r} 67 \\ \times 11 \\ \hline 67 \\ 67 \times \\ \hline 757_8 \end{array}$$

$$\begin{array}{r} 8 \overline{) 37} 4 \\ \underline{-32} \\ 5 \end{array}$$

Ex: $10532_8 \times 57_8$

$$\begin{array}{r} ^321 \\ 1053.2 \\ \times 57 \\ \hline 174556 \\ 53302 \times \\ \hline 627576_8 \end{array}$$